PLATFORM BUILT AT

lifepal

PRESENTED TO

QUILTER

CHIEF DATA OFFICE

DATA ENGINEERING · FIRESIDE CHAT · LIFEPAL TO QUILTER

Nexus

Governed data products on Databricks within Azure

Four years as technical lead and domain owner of the data platform at Lifepal, a regulated insurance marketplace, presented in the stack Quilter's Chief Data Office runs: Delta Lake on ADLS Gen2 under Unity Catalog, Azure Data Factory and Structured Streaming for movement, Data Vault and Kimball across the layers, and one certified definition of every measure served to Fabric and Power BI. It covers architecture, measured outcomes and unit cost, and every figure traces to a run or a query.

PRESENTED BY

R. Firman Insan Muhammad (VIM)
Lead Data Engineer · London

TO

Data Engineering Manager
Chief Data Office, Quilter plc

ROLE

Lead Data Engineer · Level 4
Southampton or London · new role

STACK

Azure Databricks · Delta · Unity Catalog
ADF · PySpark · Lakeflow · Fabric · Power BI

CONTENTS

Contents

I'd like to start with results and people, then go into the technical detail as far as you'd like.

PART 1	**Results and fit**	What I delivered, measured honestly, how it maps to this role, and how I work: lead and serve.	03–05
PART 2	**Serving the business**	The departments I serve (Finance, Marketing, HR, Executive, Product), the data contract every conversation ends in, the rules I hold with all of them, and Consumer Duty outcomes.	06–09
PART 3	**Team and delivery**	How the team ships: the delivery gate, and First Light, the playbook for urgent demand and reduced capacity.	10–15
PART 4	**Platform and governance**	The technical evidence starts here: platform shape, layers, the four Unity Catalog controls, security, retention and GDPR.	16–23
PART 5	**Data flow and modelling**	Contract-driven ingestion, safe replay, Data Vault and Kimball, quality at the boundary, one metric definition, and Fabric.	24–34
PART 6	**AI and cost**	Agents with governed tools, and spend managed as an engineering parameter.	35–36
CLOSING	**First 90 days and summary**	How I would join the domain, the questions I would ask, and the evidence in one page.	37–38

How to read it. Parts 1–3 are about people and outcomes: the business, the departments and the team. Parts 4–6 are the technical evidence behind them, to go as deep as you like. Every figure is from Lifepal and traces to a run or a query.

Limitations are marked in amber wherever the mechanism differs from what I ran in production, or where my depth is working-level. Page 38 summarises them.

Code and a working demo: **github.com/rfim/nexus-quilter** · **rfim.github.io/nexus-quilter**

Measured outcomes

These are the five numbers I'm proudest of from Lifepal. I've put the measurement basis next to each one, so you can ask me how I got there.

6h → 90m

Nightly batch wall-clock. Median of 30 runs either side of the change: 4.5 hours removed.

99.7%

Reduction in reporting latency: 24 h → under 5 min, on Kafka + Debezium CDC.

$11k

Monthly platform cost against a $19k projection. Held for four consecutive quarters.

−60%

Ad-hoc report requests. 90-day trailing count: 214 → 86. Data-trust escalations fell by the same share.

512

Automated quality assertions across 47 sources in CI/CD. Sustained DQ ≥ 0.98.

ATTRIBUTION OF THE BATCH-WINDOW RESULT

- **Materialisation strategy, about 55%.** Incremental merge replaced full rebuilds. Each night now recomputes the last 72 hours instead of four years of history. The core fact alone went from 2 h 14 min to 11 min (page 31).
- **Compute isolation, about 30%.** Ingestion, transformation, BI and ML each hold their own compute under their own cluster policy, so none of them queues behind another. Measured as the fall in mean job queue time.
- **Spark tuning and contract enforcement at ingestion, about 15%.** Partition pruning, AQE, broadcast joins and skew handling, plus rejection of malformed payloads before they land, which removed the re-run cycles that used to push the window past its budget.

WHY THE BASIS MATTERS MORE THAN THE NUMBER

- A batch-window figure with no run count behind it is a claim. Thirty runs either side of one change is a measurement you can reproduce from the job history.
- The 72-hour lookback is the p99.5 of observed late-arrival delay. It comes from the data; nobody picked it as a comfortable round number.
- The cost figure is monthly spend against an agreed ceiling, held across four quarters. It is a four-quarter average, so one good month doesn't flatter it.

On scale. Quilter oversees £157.4bn of customer investments. Lifepal is a mid-size insurtech: hundreds of millions of records a day under a $19k-a-month ceiling. The scale is different; the methods (contracts, layer rules, cost attribution) are the same.

PART 1 · FIT

How my experience maps to the role

I went through the job description line by line. Here is where I've done each part before, and where in this pack we can talk about it.

WHAT THE ROLE ASKS FOR	WHAT I HAVE DONE, AT LIFEPAL UNLESS STATED	PAGES
Technical leadership & domain ownership	Primary engineering authority for the platform across the Finance, Actuarial, Marketing and Product domains. I lead solution design, review code and architecture, and record significant decisions as ADRs. I also own the team's standards for pipelines, modelling, testing and security.	05 · 10 · 16 · 24
Engineering delivery	Batch and streaming on Databricks within Azure: ADF ingestion, Workflows and Airflow orchestration, Delta on ADLS Gen2. Infrastructure is Terraform, delivery runs on Azure DevOps and GitLab CI/CD, and production has had zero manual changes.	05 · 10 · 24–28
Data modelling across layers	Kimball star schemas with conformed dimensions and SCD2 in gold, built in PySpark on Delta with contracts. Data Vault is shown here as I would apply it to a multi-business estate.	17 · 29–31
Quality, SLOs, contracts, observability	500+ automated quality checks (Great Expectations and model tests) in CI. Every certified dataset carries a freshness SLO with breach alerting and 2-sigma volume anomaly detection. Root-cause analysis ends in a durable fix.	05 · 22 · 32
Governance, security, GDPR, retention	Lineage from source to report, a glossary with named owners, column masking, role- and row-level access, retention policies and Key Vault secrets. Aligned with GDPR and FCA-adjacent audit requirements.	18–23
Mentoring & capability	I line manage 3–5 engineers and promoted two to senior. Onboarding went from four weeks to two. I have also taught Databricks, Spark and dimensional modelling to 100+ engineers at Hacktiv8.	05 · 10
Serving the business	Finance, Marketing, HR, Executive and Product each own their definitions, agreed in writing before any code. Requests are challenged where the result would not be trusted. Ad-hoc requests and data-trust escalations both fell 60%.	05 · 06–09 · 33
Leading under pressure	First Light, my playbook for urgent demand and reduced capacity: ship the smallest truth that changes today's decision, keep it reversible, name the debt, and protect the team's focus.	10–15
Consumer Duty	The customer at the heart of the data roadmap: outcome MI built as certified data products that refresh on a schedule.	05 · 09

How I work: lead and serve

If I joined your team, here's how my days would go. Pick any moment, make the call yourself, and we can talk through what I'd do and why.

LEAD

The team and the platform

1 **Design it** · TECHNICAL LEADERSHIP
A Consumer Duty request arrives. I lead the design, one decision at a time, and write it up as an ADR.

2 **Ship it** · ENGINEERING DELIVERY
Finance wants net flows by channel. I take it from the first question to production, through the CI gate.

3 **Grow the team** · MENTORING
A new engineer's first pull request. I review it so they learn, without blocking them.

SERVE

The business and its clients

4 **Work with the business** · CROSS-FUNCTIONAL
A week of requests from Finance, Marketing, HR, Executive, Product and the partner teams.

5 **Keep it running** · OPERATIONAL EXCELLENCE
Monday, 07:02, on call. A board number is £250 out. I find out why before 09:00.

6 **Protect clients** · REGULATORY
Vulnerable clients wait twice as long to transfer out. I show it without exposing anyone.

7 **Keep it live** · BATCH AND REAL-TIME
Ops want today's flows now. I stream them, then check them against the books.

8 **Let them ask** · SELF-SERVICE AND AI
The semantic layer becomes a chatbot: certified answers, the asker's permissions, sources cited.

The foundation: the live lakehouse. Every chapter runs on the same Delta tables my Databricks Python built, opened in the browser with the same grants, masks and row filters, so the numbers agree from chapter to chapter. It runs in the browser, not on Databricks: DuckDB replays the Delta log. Start here: **rfim.github.io/nexus-quilter/simulate.html · open the live lakehouse**

The departments I serve

These are the teams I work with most. Each one asks a different question, so I work with each a little differently.

DEPARTMENT	WHAT THEY ASK FOR	WHAT I BUILD FOR THEM	HOW I WORK WITH THEM	AT QUILTER
Finance	Revenue, commission and cost figures they can sign off and reconcile.	Point-in-time facts, automated daily reconciliation, and restatement evidence before month-end.	Finance owns the definition; I own the pipeline. Definitions are agreed in writing before any code: at Lifepal, a "sale" took four workshops and the MERGE an afternoon.	Fee revenue, AUA and net flows; adviser charges reconciled daily.
Marketing	Campaign performance quickly enough to act on.	Certified funnel measures (conversion, cost per sale) by channel in the metric layer, with self-service on top.	Start from the decision the campaign needs. Repeat questions go to self-service. Consent and personal data are masked by default.	Client and adviser acquisition; testing whether communications are understood (Consumer Duty).
HR	Headcount, attrition and workforce planning.	Small, tightly scoped marts with personal data masked and row-level access by role.	This is the most sensitive data in the business, so access is designed with HR and the DPO before anything is built. Leaders get aggregates; only HR sees records.	Colleague and adviser workforce data, with special-category data handled under GDPR.
Executive	One number per question, and confidence that it is right.	A short set of certified KPIs, each with its owner, grain and freshness stated, so every deck and dashboard shows the same number.	Short written answers, and bad news early with an option attached. Trade-offs are explained in terms of cost, risk and time.	Board MI and the annual Consumer Duty assessment (page 09).
Product	What users do, fast enough to shape the next release.	Event and CDC streams under five minutes (from 24 hours), and funnel and feature data in gold.	Embedded with product teams. Event schemas are agreed as contracts, so a new field is reviewed before it reaches a dashboard.	Platform journeys, switches and transfers; product governance and target-market checks.

Data contracts. Every conversation with a department ends in a data contract: their definition, grain, freshness, quality rules and access, written as YAML and turned into tests (next page). They own the definition; I own the platform that keeps it true.

Agreements become data contracts

This is something I've always done: every agreement with a department ends up as a data contract in YAML, and the tests come straight from it.

CONTRACTS/PRODUCTS/FINANCE_NET_FLOWS.YML — YAML · DATA CONTRACT

```yaml
product:     finance_net_flows_daily
department:  Finance
owner:       finance-reporting-lead@example.com  # signs off the definition
engineer:    vim@example.com                     # owns the pipeline
definition:  >
  Net Flows = sum(amount_gbp) over contributions (positive), withdrawals and
  fees (negative). Switches between funds and income are excluded.
grain:       [legal_entity, wrapper_code, trade_date]

freshness:
  column:    trade_date
  max_lag_days: 1          # ready by 06:30 Europe/London, business days

columns:
  - name:  legal_entity
    tests: [not_null, {accepted_values: [affluent, hnw]}]
  - name:  wrapper_code
    tests: [not_null, {accepted_values: [ISA, JISA, PENSION, GIA, BOND]}]
  - name:  net_flows_gbp
    tests: [not_null]
reconcile:
  against:   finance_ledger
  match_on:  [trade_date, wrapper_code]
  measure:   net_flows_gbp
  ledger_measure: ledger_net_flows_gbp
  tolerance_gbp: 0.01
access:
  read:      [grp_finance, grp_exec_mi]
change:
  breaking:  owner_approval   # blast radius posted on the PR
```

See the code: **github.com/rfim/nexus-quilter/contracts/products/finance_net_flows.yml**

CI/DERIVE_FROM_CONTRACTS.PY — PYTHON

```python
for c in contracts.load_products():
    write(f"{c['product']}_expectations.py", compile_expectations(c))
    write(f"{c['product']}_governance.sql", compile_governance(c))
    results = run_checks(spark, c)      # SQL from compile_checks(c)
    log_results(c["product"], results)  # kept, and shown on the site
```

See the code: **github.com/rfim/nexus-quilter/src/nexus/contracts.py**

WHAT THE YAML TURNS INTO

- **Tests.** Column tests, grain uniqueness and reconciliation against the ledger, run in CI and on every load.
- **A freshness SLO** with an alert routed to the named owner, reported as attainment over time.
- **Grants and masks.** Who may read the product, and which columns are protected.
- **Change control.** A breaking change needs the owner's approval on the pull request.

WHY DEPARTMENTS ACCEPT IT

- They read a short YAML file. The diff shows who agreed what, and when.
- A disagreement becomes a pull request with options, so there is a record.

Same mechanism end to end. Source contracts drive ingestion (page 24), product contracts drive tests and access. That is how 500+ checks reached CI at Lifepal.

How I handle requests

Whoever is asking, I follow the same few rules. The main one is no surprises.

PARTNERS WHO KEEP IT SAFE	WHAT THEY NEED	HOW I WORK WITH THEM	EVIDENCE, AT LIFEPAL
Architecture, platform, governance	Solutions inside enterprise standards	Significant decisions are written up as ADRs before the build. Standards are enforced as CI checks.	Zero manual production changes.
DPO, risk, compliance	Evidence they can check	Access is a business decision recorded in a governed table. Retention and erasure calls are theirs; engineering turns each one into a reviewed run.	Aligned with GDPR and FCA-adjacent audit.
Engineering manager, product owners	Predictable delivery and visible trade-offs	One weekly forum for incidents, delivery and cost together. Risks and dependencies are listed on the roadmap up front.	$11k against $19k, held for four quarters.

WHEN A REQUEST ARRIVES

- **Start from the decision.** What will you do differently with this number?
- **Check the certified measures first.** Most requests are already answered by one.
- **New definitions go into the metric layer first**, with a named owner, and into the report second.
- **Explain trade-offs in their terms**: freshness, cost and risk.

WHEN THE ANSWER IS NO, OR NOT YET

- **A reason and an alternative, every time.** "Not every 15 minutes, but six-hourly meets your SLA at a fraction of the cost."
- **Definition disputes go to the data owner**, with the options written down.
- **Bad news early and in writing.** A blameless incident write-up, published where the business can read it, the same week.
- **Cost gets a name on it.** Publishing spend per team changed behaviour with no policy change at all.

> **Result at Lifepal.** Ad-hoc requests fell from 214 to 86, and data-trust escalations fell by 60%.

Consumer Duty reporting

An idea I'd like to discuss: Consumer Duty outcome reporting built the same way as everything else on the platform.

OUTCOME	THE QUESTION	CERTIFIED MEASURE (ILLUSTRATIVE)	GRAIN	SOURCE DOMAINS
Products & services	Is each client in a product that fits its target market?	Share of clients outside target market, by product and channel	Client × product, monthly	Platform, advice, product governance
Price & value	Does the total cost hold up against the value received?	Total cost of ownership ÷ AUA, by cohort and wrapper	Client cohort × wrapper, quarterly	Platform charges, fund OCF (SCD2, page 30)
Consumer understanding	Did communications land with the people they were for?	Engagement and query rates following key communications	Communication × segment, monthly	Comms, servicing
Consumer support	Can clients act as easily as they can join?	Time to withdraw or transfer out vs time to onboard	Request × channel, weekly	Servicing workflow, platform events

WHAT ENGINEERING CONTRIBUTES

- **Every outcome measure is a data product.** It has a contract, an owner, a freshness SLO and a metric-view definition, so the annual board assessment is a query over a governed history.
- **Outcomes for vulnerable clients, without exposing them.** The vulnerability flag is classified and masked at row level (page 19), but aggregates by flag stay available, so differential outcomes are measurable without anyone seeing who is vulnerable. **See it live in chapter 6**.

WHY I RAISE IT IN A DATA ENGINEERING INTERVIEW

- The role description puts the customer at the heart of the data roadmap. In practice that means outcome measures get the same engineering standard as financial reporting: tested, lineaged and reproducible.
- Foreseeable harm shows up in data before it shows up in complaints. A monitored outcome measure gives early warning.

> **Note.** This page is a proposal; I haven't built this before. The measures are illustrative, and the real definitions belong to Quilter's Consumer Duty and product governance owners. What I would bring is the machinery that makes their definitions durable.

How the team works

How I run the team day to day. I still review code every day; it keeps my prioritisation honest.

AZURE-PIPELINES.YML — AZURE DEVOPS

```yaml
# azure-pipelines.yml · the gate a new joiner meets on day one
trigger: [main]
pr: [main]
stages:
- stage: validate
  jobs:
  - job: checks
    steps:
    - script: ruff check . && mypy src/ pipelines/
    - script: pytest tests/ -q             # transforms are unit tested
    - script: terraform -chdir=infra plan -input=false
    # bundle must be valid for every target
    - script: databricks bundle validate -t test
    - script: databricks bundle validate -t prod
    # build only what changed, deferring the rest to production state
    - script: databricks bundle run -t test nexus_model_tests
    # posts the transitive downstream set and its owners onto the PR
    - script: python ci/blast_radius.py --pr $(System.PullRequest.PullRequestId)
    # a certified metric moving > 0.5% fails the merge
    - script: python ci/metric_diff.py --threshold 0.005

- stage: deploy
  dependsOn: validate
  condition: and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/main'))
  jobs:
  - deployment: prod
    environment: nexus-prod              # approval gate lives here
    strategy:
      runOnce:
        deploy:
          steps:
          - script: databricks bundle deploy -t prod
```

OPERATING CADENCE

- One weekly meeting covers **incidents, delivery and cost together**. They are the same conversation, and separating them hides the trade-offs.
- Every pull request is reviewed within one working day, by me or a named second. Significant design decisions are written up as ADRs.
- Fortnightly one-to-ones with a written record of the capability being developed.
- A blameless write-up for every incident, published where the business can read it.
- **Documentation is part of the definition of done.** An undescribed model does not merge, which is how documentation survives a busy quarter.

WHAT IT PRODUCED AT LIFEPAL

- **Two engineers promoted to senior**, on stretch assignments with written success criteria and feedback the same week.
- **Onboarding cut from four weeks to two**: pairing from day one, standardised model documentation, and CI that catches defects before a reviewer has to.
- **Zero manual production changes.** Terraform for infrastructure, bundles for workloads, one deployment path.

The Principle of First Light

My playbook for the weeks when everything is urgent, requirements are unclear, or half the team is out.

FIRST LIGHT

"When navigating uncertainty, noise or constrained capacity, do not seek perfection. Deliver the smallest illuminating truth today to guide tomorrow's direction."

Operating meaning Ship the smallest truth that changes a decision today. Keep the choice reversible, name the debt, and let tomorrow's evidence improve the build.

"Ship the candle. Build the lighthouse later."

FIRST LIGHT IN A REGULATED FIRM

Controls stay on

Classification, masking and the source contract are code, so they take minutes (pages 19, 20, 24).

Every change is reviewed

Every candle goes through the same gate: tests, bundle validation, blast radius and metric diff (page 10).

Debt is written down

Debt taken during First Light is logged with an owner and a revisit date, and reviewed in the weekly forum.

Three beliefs

The three ideas behind it.

1

Light over perfection

Perfect architecture delivered late illuminates nothing. Favour the simplest reversible solution that gives immediate clarity. First light beats full sun tomorrow.

Where it shows in this pack. Bounded triggers over continuous streaming; incremental merge over a nightly rebuild (pages 26, 31).

2

Signal over noise

Filter requests and technical debates by the decision they enable today. Ask what single core decision this data or code needs to support.

Where it shows in this pack. Start from the decision; check certified measures before building new reports (pages 06, 33).

3

Sanctuary for execution

Leadership shields the team from ad hoc chaos, capacity swings and context switching. One clear focus produces real velocity.

Where it shows in this pack. One primary commitment per engineer; one weekly forum for incidents, delivery and cost (page 10).

Why this fits Quilter's values. Embrace challenge: aim high, but ship today's truth first. Always curious: let tomorrow's evidence improve the build. Stronger together: protect the team's focus so it can deliver.

First Light in practice

What it changes in practice, for decisions, requests and the team.

PILLAR	PRINCIPLE	ACTION IN PRACTICE
Technical decisions	**Reversibility first**	If a decision is a two-way door, take the fastest path to production today. Log the technical debt explicitly and revisit it after First Light.
Stakeholder requests	**Minimum Viable Truth**	Deliver the raw insight, a simple summary or the core metric today to unblock the business decision. Don't build a full pipeline for an ad hoc question.
Team capacity	**Single-task focus**	When capacity drops, cut scope and keep quality. Each engineer keeps one primary sprint commitment so attention stays protected.
Regulated change	**Controls are the fast path**	Anything touching client data, access or regulatory MI goes through classification, masking and the contract by default. Because they are code, they add minutes.

Operating rule. Fast path, explicit debt, protected focus. Trade-offs are written down early so they can guide the next build.

Added for a regulated firm. I added the fourth pillar for a regulated firm. In a wealth business, skipping a control just creates remediation work later.

Sorting data requests

How I sort incoming requests, so the team can focus on what matters.

	URGENT	NOT URGENT
IMPORTANT	DO FIRST **Critical path and MVT only** ■ Critical-path pipeline fixes ■ Minimum Viable Truth for today's executive or regulatory decision ■ Fix or answer today, polish later	SCHEDULE **Core sprint backlog** ■ Scalable modelling: vault and Kimball ■ Data governance and quality ■ Technical debt logged during First Light
NOT IMPORTANT	DELEGATE **Self-service and governed agent** ■ Ad hoc requests go to certified metric views or the governed agent (page 35) ■ Routine requests get no hand-written custom SQL from engineers	ELIMINATE **Low business value** ■ Decline custom multi-page reports with no decision attached ■ Archive manual recurring data dumps

Automation first. The metric layer and the governed agent are the defensive layer against ad hoc disruption. At Lifepal, that layer is what took ad-hoc requests from 214 to 86 (page 33).

Making decisions under pressure

Three questions I ask the team to use when things get busy.

1 **Does this directly affect the critical path today?**

Yes: ship the simplest reversible build, then defer perfection.

No: push it to the backlog, or route it to self-service or the governed agent.

2 **What is the smallest answer that enables today's decision?**

Deliver a raw query, a summary metric or an existing view.

Omit polish until the decision proves it needs more.

3 **Are we making a one-way-door decision?**

Yes: escalate to the lead, and to the data owner where it applies, for an immediate call. Client data leaving the boundary, a certified regulatory measure, an access grant and a widely used schema are always one-way doors.

No: bias for action. The author decides and moves.

To our engineers

You will not carry architectural liability for pragmatic decisions made under time pressure to protect delivery. Momentum and iteration matter.

To our stakeholders

You will not get radio silence. You get the fastest path to today's truth through a minimum viable answer or self-service, while we protect long-term platform health.

To our risk partners

Speed never comes from skipping a control. It comes from controls being code.

"Ship the candle. Build the lighthouse later."

How the platform fits together

This is the whole platform on one page. I draw governance underneath, because it touches every stage.

Acquire

Platform events and CDC over Kafka-compatible streams; ADF for scheduled system extracts; Auto Loader for fund-manager, custodian and adviser-firm file drops.

`ADF · Structured Streaming`

Orchestrate

140 pipelines generated from YAML contracts at build time. Bounded retries, health rules and declared SLA budgets.

`Asset Bundles · Workflows`

Persist

Bronze and silver as Delta on ADLS Gen2, in a storage account the business owns. Private endpoints, Key Vault-backed secrets.

`Delta Lake · ADLS Gen2`

Model

Data Vault in silver to integrate sources by business key; Kimball conformed dimensions and SCD2 marts in gold.

`PySpark · Lakeflow`

Serve

Metric views define each certified measure once. Power BI and Fabric semantic models, notebooks and agents read gold only.

`Metric views · Fabric`

UNITY CATALOG · GOVERNANCE PLANE · CLASSIFY → ENFORCE → TRACE → MEASURE

Classify

Governed tags applied at landing. Client personal data and vulnerability flags are tagged when they arrive.

Enforce

Column masks and row filters bound to the tag and the legal entity, so protection is in place before the first query.

Trace

Column lineage from system tables. A schema change is gated on its transitive downstream set.

Measure

Monitoring profiles served tables on a schedule. Agent tools are governed functions.

Governing design constraint. Bronze and silver are Delta tables on ADLS Gen2 that Quilter owns. Databricks is the execution engine and the governance plane over that data. Fabric reads the same files through OneLake, so there is only one copy of the data, kept in an open format.

PART 4 · LAYER MODEL

Bronze → Silver → Gold

I keep the three layers strict: each has its own rules about what goes in and who can read it.

Bronze

`nexus.bronze`

As received, unmodified

CONTENTS

Stream events, CDC rows and file-drop records with the payload kept verbatim. No reshaping, no typing.

GRAIN

One row per source record version, keyed on merge key plus CDC sequence.

FORMAT

Delta, written by MERGE and never by INSERT. Deletion vectors on.

READ BY

Platform engineers only. No business group holds a grant.

Silver

`nexus.silver`

Integrated, historised, auditable

CONTENTS

Data Vault hubs, links and insert-only satellites, plus conformed SCD2 dimensions built from them.

GRAIN

One row per business key per change, stamped with load date and record source.

FORMAT

Delta with liquid clustering. Incremental merge on a bounded lookback.

READ BY

Analytics engineers, PySpark jobs and ML feature builds.

Gold

`nexus.gold`

Certified and served

CONTENTS

Kimball marts, metric views and data products. Every published measure is defined exactly once.

GRAIN

Atomic facts, plus aggregates such as client × wrapper × day.

FORMAT

Delta and materialised views, refreshed to a declared lag.

READ BY

Power BI and Fabric, notebooks, agent tools and the business.

Access is enforced by grants. No reporting group, agent or service principal holds a grant on bronze or silver. Data moves between layers only through a reviewed model. Masks and row filters attach at silver and gold, so bronze is unexposed by construction, which is cheaper than trying to mask it. Retention per data class is on page 23.

Unity Catalog controls

I start with these four because every later design decision has to work within them.

01 · PAGE 19

Classify

Governed tag taxonomy at the metastore. Candidate personal data with no tag is a weekly, queryable exception.

Attaches at bronze, on arrival. Swept every run.

02 · PAGE 20

Enforce

Column masks and row filters bound to the tag and the legal entity, so a new mart inherits protection with no extra deploy step.

Attaches at silver and gold. Bronze holds no business grant.

03 · PAGE 21

Trace

Column lineage from `system.access`, complemented by declared dependencies in the transformation project.

Attaches at every layer transition and at the BI boundary.

04 · PAGE 22

Measure

Monitoring profiles freshness, completeness, uniqueness and domain plausibility on a schedule.

Applies at gold. Results are stored, so trends are visible.

Why this order

Governance added after a pipeline is an audit. Governance added before it is a constraint. Every page that follows (the source contract, the layer boundaries, the metric view, the agent tools) is shaped by these four controls, so they come first. It also makes reviews quicker, because the reviewer checks a model against known controls.

Consequence for the pipeline

- A new column is classified and masked before any consumer can query it.
- A schema change is blocked until its downstream owners have been notified.
- A table that goes stale is flagged by monitoring before a user notices.
- An agent uses the caller's masks and row filters, so it sees only what the caller can see.

Tagging sensitive data

I don't trust a manual audit to find personal data, so a scheduled job looks for it every week.

GOVERNANCE/CLASSIFICATION.SQL — SQL

```sql
-- 1 · one governed tag for the metastore, created once at account
--     level with an allowed-value list (Catalog Explorer or the API):
--     nexus_pii ∈ {ni_number, date_of_birth, bank_account, email,
--                  phone, postal_address, client_id, vulnerability}

-- 2 · tags applied at the column, inherited by lineage children
ALTER TABLE nexus.silver.sat_client_details
  ALTER COLUMN national_insurance_no SET TAGS ('nexus_pii' = 'ni_number');

ALTER TABLE nexus.silver.sat_client_circumstances
  ALTER COLUMN vulnerability_flag SET TAGS ('nexus_pii' = 'vulnerability');

ALTER SCHEMA nexus.bronze SET TAGS ('data_zone' = 'restricted');

-- 3 · the coverage control, reviewed weekly: candidate personal
--     data carrying no tag. Coverage is the primary metric.
SELECT c.table_schema, c.table_name, c.column_name
FROM   system.information_schema.columns c
LEFT JOIN system.information_schema.column_tags t
       ON  t.catalog_name = c.table_catalog
       AND t.schema_name  = c.table_schema
       AND t.table_name   = c.table_name
       AND t.column_name  = c.column_name
       AND t.tag_name     = 'nexus_pii'
WHERE  c.table_catalog = 'nexus'
  AND  t.tag_value IS NULL
  AND  c.column_name RLIKE concat('(?i).*(email|phone|mobile|ni_?no|nino',
         '|dob|birth|sort_code|account_no|iban|postcode|address',
         '|client_id|vulnerab).*');
```

WHY THIS COMES FIRST

- Every later control (masking, row filtering, lineage, quality monitoring) is keyed on a tag. If tag coverage is incomplete, the governance layer degrades **silently**, so coverage is tracked as the primary metric.

OBSERVED AT LIFEPAL

- Automated classification found **118 sensitive columns across 47 schemas**, 23 of which had never been tagged by hand.
- Median time from a new column appearing to it being tagged fell from "next audit" to **under 24 hours**.

What changes for a wealth estate. Client records carry NI numbers, bank details and dates of birth, often across several generations of one family. Vulnerability flags under Consumer Duty need their own tag value: masked at row level, but still aggregable for outcome MI (page 09).

Limitation. Pattern detection has false negatives. The sweep is a backstop. The primary control is that a source cannot be onboarded until its contract declares the tag.

Masking and row filters

Once a column is tagged, it's masked by default. If one slips through, it shows up in a weekly report.

GOVERNANCE/POLICIES.SQL SQL

```sql
-- 1 · one masking function for the whole estate
CREATE OR REPLACE FUNCTION nexus.governance.mask_pii(val STRING)
RETURN CASE
  WHEN is_account_group_member('grp_dpo')             THEN val
  WHEN is_account_group_member('grp_client_services') THEN val
  WHEN is_account_group_member('grp_analyst')
       THEN sha2(lower(trim(val)), 256)
  ELSE '***'
END;

-- 2 · bound to the column; every consumer inherits it
ALTER TABLE nexus.silver.sat_client_details
  ALTER COLUMN email_address SET MASK nexus.governance.mask_pii;

-- 3 · legal-entity scope read from a table the business owns:
--     Affluent and High Net Worth see their own clients only
CREATE OR REPLACE FUNCTION nexus.governance.entity_filter(entity STRING)
RETURN is_account_group_member('grp_group_mi')
    OR EXISTS (SELECT 1 FROM nexus.governance.group_entity_grant g
               WHERE is_account_group_member(g.group_name)
                 AND g.legal_entity = entity);

ALTER TABLE nexus.gold.fct_holding_valuation_daily
  SET ROW FILTER nexus.governance.entity_filter ON (legal_entity);
```

Where this differs from my previous platform. Unity Catalog ABAC policies can bind a mask to the governed tag itself. Until that is the standard in the estate, the binding is a deploy step, so a CI check on every pull request confirms each tagged column has its mask.

THE PROPERTY BEING BOUGHT

- Protection doesn't depend on the engineer building the next mart remembering it. A column classified under control 1 is masked from the moment it exists, with **no deployment step in the reader's path and no reviewer in the loop**.
- `group_entity_grant` is maintained by the business. An access change is recorded as a business decision in a governed table.

THREE WEEKLY CONTROLS

- **Untagged-column sweep.** Candidate personal data carrying no tag.
- **Policy coverage.** Gold relations holding entity-scoped client data with no row filter attached.
- **Group concentration.** Any group granted to more than one persona is split before it becomes the group everybody is in.

Serving MI without widening access. Power BI reads gold through the same masks and filters as a human analyst. There is no elevated service principal behind a dashboard, which is a common way governance gets bypassed.

Lineage and impact analysis

The question I ask before any change: if this column changes, what breaks, and who needs to know first?

GOVERNANCE/LINEAGE.SQL — SQL

```sql
-- 1 · column lineage, captured by the platform. No external catalog.
SELECT source_table_full_name, source_column_name,
       target_table_full_name, target_column_name,
       max(event_time) AS last_observed
FROM   system.access.column_lineage
WHERE  event_time > current_timestamp() - INTERVAL 30 DAYS
  AND  source_table_full_name = 'nexus.silver.sat_fund_share_class'
  AND  source_column_name     = 'ongoing_charge_pct'
GROUP BY ALL;

-- 2 · transitive downstream set, resolved to a notification list
WITH RECURSIVE downstream (obj, hop) AS (
  SELECT target_table_full_name, 1
  FROM   system.access.table_lineage
  WHERE  source_table_full_name = 'nexus.silver.sat_fund_share_class'
  UNION ALL
  SELECT l.target_table_full_name, d.hop + 1
  FROM   system.access.table_lineage l
  JOIN   downstream d ON l.source_table_full_name = d.obj
  WHERE  d.hop < 8
)
SELECT d.obj, min(d.hop) AS hops, any_value(o.owner_email) AS notify
FROM   downstream d
JOIN   nexus.governance.object_owner o ON o.object_name = d.obj
GROUP BY d.obj ORDER BY hops;
```

FOUR APPLICATIONS

- **Continuous integration.** A pull request that drops or retypes a column posts its transitive downstream set, and the owners to notify, onto the review. Nobody has to answer "does anything depend on this?" from memory.
- **Incident triage.** At Lifepal, median time to trace a reported discrepancy to its source relation fell from hours to **under five minutes**.
- **Subject access and erasure.** Every downstream location of a tagged client column can be listed with a query, which is what GDPR requires (page 23).
- **Board and Consumer Duty MI.** A figure in an outcomes report traces to its source tables, so a challenge can be answered quickly.

> **Limitation.** Platform lineage stops at the workspace boundary. Lineage into Power BI is rebuilt from dataset metadata. That is enough to route a change notice, but not enough to present as a formal control, and I would rather say so than imply the graph is complete.

Data quality monitoring

I keep every quality measurement in a table. That way thresholds come from real history, and anyone can look back at last Tuesday.

GOVERNANCE/MONITOR.PY — PYTHON

```python
from databricks.sdk import WorkspaceClient
from databricks.sdk.service.catalog import (
    MonitorTimeSeries, MonitorMetric, MonitorMetricType, MonitorCronSchedule)
from pyspark.sql import types as T

w = WorkspaceClient()

# a domain metric: share of holdings with no price on the valuation date
unpriced = MonitorMetric(
    name="unpriced_holding_ratio",
    definition="sum(CASE WHEN unit_price IS NULL THEN 1 ELSE 0 END) / count(*)",
    type=MonitorMetricType.CUSTOM_METRIC_TYPE_AGGREGATE,
    input_columns=[":table"],
    output_data_type=T.StructField("output", T.DoubleType()).json())

w.quality_monitors.create(
    table_name="nexus.gold.fct_holding_valuation_daily",
    assets_dir="/Workspace/Shared/nexus/monitoring",
    output_schema_name="nexus.monitoring",
    time_series=MonitorTimeSeries(timestamp_col="valuation_date",
                                  granularities=["1 day"]),
    custom_metrics=[unpriced],
    schedule=MonitorCronSchedule(quartz_cron_expression="0 0 6 * * ?",
                                 timezone_id="Europe/London"))
```

GOVERNANCE/ALERTING.SQL — SQL

```sql
-- one relation carries every measurement; alerting reads only this
SELECT window.start AS measured_at, column_name,
       num_nulls, distinct_count, unpriced_holding_ratio
FROM   nexus.monitoring.fct_holding_valuation_daily_profile_metrics
WHERE  window.start > current_timestamp() - INTERVAL 7 DAYS;
```

DIVISION OF LABOUR

- **Expectations, before load.** Reject malformed batches at the ingestion boundary, where the fix is cheapest (page 32).
- **Monitoring, after load.** Profile served products continuously and independently of the pipeline that built them. A pipeline that "succeeded" while writing wrong numbers still gets caught.
- **Freshness SLOs.** Every certified dataset has one: runs landing inside the target ÷ scheduled runs, reported as attainment over time.

WHY KEEP THE MEASUREMENTS

- A profile is a time series, so a change in data behaviour is visible **before** it becomes an incident.
- At Lifepal, freshness breaches were caught a median of **41 minutes earlier** than under the previous alerting.
- The business owner can query the same relation, which ends any argument about whether a valuation "looked wrong last Tuesday".

Retention and GDPR erasure

How I handle GDPR, retention and security in practice, with evidence you can query.

GOVERNANCE/RETENTION.SQL — SQL

```sql
-- retention is data, owned by the business and read by the platform
CREATE TABLE nexus.governance.retention_schedule (
  data_class      STRING,   -- client_record | transaction | marketing ...
  table_full_name STRING,
  retain_for      STRING,   -- set by the DPO and compliance
  trigger_event   STRING,   -- e.g. relationship_end
  basis           STRING,   -- regulatory | contractual | consent
  owner           STRING
);
```

GOVERNANCE/ERASURE.PY — PYSPARK · DELTA

```python
# erasure: every location of the client, found by tag + lineage (page 21)
for t in tables_tagged("nexus_pii", "client_id"):
    if retention_hold(t, data_class_of(t)):        # regulatory basis wins
        restrict(t, approved_ids)                  # restrict processing instead
        continue
    spark.sql(f'''DELETE FROM {t} WHERE client_id IN
                  (SELECT client_id FROM nexus.governance.erasure_queue
                   WHERE approved)''')
    spark.sql(f"REORG TABLE {t} APPLY (PURGE)")    # rewrite files with DVs

# VACUUM after the declared history window removes the bytes physically
```

SECURE BY DESIGN ON AZURE

- **Network.** VNet-injected workspaces with secure cluster connectivity. Private endpoints to ADLS Gen2 and Key Vault, and no public storage endpoints.
- **Identity.** Unity Catalog reaches storage through an Access Connector managed identity. People and jobs authenticate through Entra ID groups; nobody holds a storage key.
- **Secrets.** Key Vault-backed secret scopes only. A credential in a notebook or a repository fails CI.
- **Retention.** Delta history windows are set per layer, and record retention per data class comes from the schedule on the left. Both are written down as configuration.

> **Limitation.** Erasure and regulatory retention conflict by design. The schedule records which one wins for each data class, and that call belongs to the DPO and compliance. Engineering's job is to make either outcome a single reviewed, auditable run.

The source contract

When a team wants a new source, this file is all they change. No new job code, no new notebook.

CONTRACTS/SOURCES.YML — YAML

```yaml
- pipeline_id:    nexus__ingest__platform__transaction
  domain:         platform
  entity:         transaction
  segment:        affluent
  owner:          platform-data@example.com

  source:
    type:         kafka          # kafka | cdc | autoloader | adf
    bootstrap:    eh-nexus-prod  # Event Hubs, Kafka endpoint
    topic:        platform.transactions.v2
    starting:     latest
    max_offsets:  500000

  target:
    layer:        bronze         # bronze | silver | gold
    table:        nexus.bronze.platform_transaction
    format:       delta
    write_mode:   merge
    merge_keys:   [transaction_id]
    ordering_key: cdc_lsn        # highest sequence wins
    cluster_by:   [trade_date, account_id]

  sla:
    trigger:        "5 minutes"
    budget_minutes: 12

  quality:
    suite:        platform_transaction  # expectation set, page 32
    on_failure:   quarantine     # quarantine | fail | warn
    dq_threshold: 0.98

  governance:
    tag_policy:   nexus_pii
    data_class:   transaction    # → retention_schedule, page 23
    legal_entity: from_column
```

See the code: **github.com/rfim/nexus-quilter/contracts/sources/sources.yml**

WHAT EACH BLOCK DRIVES

- `source` → the stream reader, its parallelism and the deserialiser.
- `target.layer` → the destination, the job tag and which grants apply.
- `merge_keys` / `ordering_key` → the dedup window and MERGE predicates on page 27. **Deduplication is declared here once, for every source.**
- `sla` → the stream trigger and the job health rule. Refresh frequency is a stated SLA that someone owns.
- `quality` → the expectation set and what happens on failure.
- `governance` → tag policy, retention class and the legal-entity column the row filter keys on.

> **Who reviews what.** The data owner in the business domain reviews the contract; engineering reviews the factory that consumes it. That split is what lets the framework survive a change of engineer, and it is how a domain lead scales across more sources than they can personally read.

lifepal

Pipelines generated from contracts

At Lifepal, 140 jobs came out of one module like this. When we changed the retry policy, it was one pull request for all 140.

BUNDLE/GENERATE_JOBS.PY — PYTHON · ASSET BUNDLES

```python
CONTRACTS = yaml.safe_load(Path("contracts/sources.yml").read_text())

def job_for(cfg: dict) -> dict:
    src, tgt, sla = cfg["source"], cfg["target"], cfg["sla"]
    return {
      "name": cfg["pipeline_id"],
      "tags": {"layer": tgt["layer"], "domain": cfg["domain"],
               "owning_team": cfg["owner"], "workload": "ingest"},
      "queue": {"enabled": True},
      "max_concurrent_runs": 1,
      # the platform enforces the SLA budget
      "health": {"rules": [{"metric": "RUN_DURATION_SECONDS",
                            "op": "GREATER_THAN",
                            "value": sla["budget_minutes"] * 60}]},
      "email_notifications": {"on_failure": [cfg["owner"]]},
      "tasks": [{
          "task_key": "ingest",
          "job_cluster_key": "ingest_shared",
          "spark_python_task": {"python_file": "src/ingest.py",
                                "parameters": ["--contract", cfg["pipeline_id"]]},
          "max_retries": 4, "min_retry_interval_millis": 120000,
          "retry_on_timeout": True}],
    }

# 140 jobs, one module under review, one bundle deployed per target
Path("resources/jobs.yml").write_text(yaml.safe_dump(
    {"resources": {"jobs": {c["pipeline_id"]: job_for(c) for c in CONTRACTS}}}))
```

See the code: **github.com/rfim/nexus-quilter/databricks.yml**

PROPERTIES OF THE APPROACH

- **Single point of change.** A retry-policy change reaches all 140 pipelines in one reviewed diff, with no per-pipeline drift.
- **Bounded onboarding cost.** A new feed is a 12-line contract change. Median time from request to first load is **under one day**.
- **Ownership is declared.** Every job carries an owner, a cost tag and an SLA budget, so every alert goes to a named engineer.
- **Environments are the same code.** Dev, test and production are bundle targets deployed from Azure DevOps, so nothing can be fixed in prod only.

> **Prior state, for honest comparison.** Before this, there were 38 hand-maintained pipeline modules with three different retry idioms, two alerting paths and no queryable record of who owned which table. That is the usual state of an estate built by several teams over several years, and consolidating it is the standards work this role describes.

Streaming and file ingestion

One reader handles event streams, CDC and file drops. The contract tells it which; the code doesn't care about source names.

SRC/INGEST.PY — PYSPARK

```python
from pyspark.sql import functions as F

def read_source(spark, cfg):
    src = cfg["source"]
    if src["type"] in ("kafka", "cdc"):
        return (spark.readStream.format("kafka")
                .option("kafka.bootstrap.servers", bootstrap(src))
                .option("kafka.security.protocol", "SASL_SSL")
                .option("kafka.sasl.mechanism", "PLAIN")
                .option("kafka.sasl.jaas.config", jaas(src))  # Key Vault scope
                .option("subscribe", src["topic"])
                .option("startingOffsets", src["starting"])
                # bounds the batch, so a backlog cannot blow the SLA budget
                .option("maxOffsetsPerTrigger", src["max_offsets"])
                .load()
                .select(F.from_avro(F.col("value"), schema_of(cfg)).alias("r"),
                        F.col("timestamp").alias("_source_ts"))
                .select("r.*", "_source_ts"))

    if src["type"] == "autoloader":   # fund-manager and custodian drops
        return (spark.readStream.format("cloudFiles")
                .option("cloudFiles.format", src["file_format"])
                .option("cloudFiles.schemaLocation", schema_path(cfg))
                .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
                .load(src["landing"]))    # abfss://landing@.../<feed>/

def run(spark, cfg):
    df = read_source(spark, cfg).withColumn("_ingested_at",
                                            F.current_timestamp())
    (df.writeStream
       .foreachBatch(lambda b, i: upsert_bronze(b, i, cfg))   # page 27
       .option("checkpointLocation", checkpoint(cfg))
       .trigger(processingTime=cfg["sla"]["trigger"])
       .start())
```

See the code: **github.com/rfim/nexus-quilter/src/nexus/bronze.py**

THREE DECISIONS WORTH DEFENDING

- **Bounded triggers over continuous.** `maxOffsetsPerTrigger` plus a five-minute trigger keeps each micro-batch inside its budget, so a backlog slows things down gradually and never produces one huge batch that misses the window.
- **Schema evolution is additive and explicit.** Auto Loader adds new columns and stops the stream, so no field is dropped silently. The restart is deliberate: a new field is a contract change and someone should see it.
- **Spark checkpoints track offsets.** The engine owns offset tracking. The previous platform did this by hand, which was one more place for recovery to go subtly wrong.

> **What this gets Quilter specifically.** A wealth estate has both shapes: intraday platform events (contributions, switches, withdrawals) and daily files from fund managers, custodians, pricing vendors and adviser back offices. One reader for both means the files get the same contract, quality gate and ownership record as the streams.

Safe re-runs with MERGE

I design every load so it can be re-run safely, whether it's a five-minute batch or a six-month backfill. It saves a lot of 7am stress.

SRC/MERGE.PY — PYSPARK · DELTA

```python
from delta.tables import DeltaTable
from pyspark.sql import Window, functions as F

def upsert_bronze(micro_batch, batch_id: int, cfg) -> None:
    keys  = cfg["target"]["merge_keys"]
    order = cfg["target"]["ordering_key"]

    # Deduplicate INSIDE the batch, so a replayed offset range
    # cannot be applied twice. This is what makes replay safe.
    w = Window.partitionBy(*keys).orderBy(F.col(order).desc())
    latest = (micro_batch
              .withColumn("_rn", F.row_number().over(w))
              .filter("_rn = 1").drop("_rn"))

    cond = " AND ".join(f"t.{k} = s.{k}" for k in keys)

    (DeltaTable.forName(spark, cfg["target"]["table"]).alias("t")
       .merge(latest.alias("s"), cond)
       .whenMatchedUpdate(                          # soft delete
           condition=f"s.{order} > t.{order} AND s.op = 'D'",
           set={"_deleted": F.lit(True), order: f"s.{order}"})
       .whenMatchedUpdateAll(condition=f"s.{order} > t.{order}")
       .whenNotMatchedInsertAll(condition="s.op <> 'D'")
       .execute())
```

See the code: **github.com/rfim/nexus-quilter/src/nexus/bronze.py**

THE ORDERING GUARANTEE

- The checkpoint commits **after** the MERGE. A failure in between costs repeated work but never leaves a gap. The asymmetry is deliberate: the dedup window neutralises a duplicate, whereas a lost offset cannot be recovered.
- At-least-once delivery from the transport becomes an at-most-once *effect* on the table. That is how exactly-once is achieved in practice.

OBSERVED AT LIFEPAL

- **24 months with no corrective DML** against a raw table. Every recovery was a re-run of the failed interval.
- Daily commission reconciliation with partners became fully automated, because a re-run could never double-count.

> **Why this matters in incidents.** After most incidents, the hard part is knowing what state the table is in. With safe replay, the run-book is one line: re-run the interval. For Quilter, the same property is what lets daily fee, adviser-charge and unit reconciliations be re-run without fear.

Delta tables and time travel

One copy of the data, readable by more than one engine. Restatements happen in wealth, so I plan for them.

DDL/GOLD_FCT_HOLDING_VALUATION_DAILY.SQL — SQL

```sql
CREATE TABLE nexus.gold.fct_holding_valuation_daily (
  valuation_sk    BIGINT NOT NULL,  account_sk BIGINT NOT NULL,
  client_sk       BIGINT NOT NULL,
  fund_sk         BIGINT NOT NULL,   -- share-class version in force
  wrapper_sk      BIGINT NOT NULL,   -- ISA | pension | GIA | bond
  valuation_date  DATE   NOT NULL,
  units           DECIMAL(28,6),    unit_price DECIMAL(18,6),
  value_gbp       DECIMAL(18,2),
  legal_entity    STRING NOT NULL)   -- keys the row filter, page 20
USING DELTA
CLUSTER BY (valuation_date, account_sk)          -- liquid clustering
TBLPROPERTIES (
  'delta.enableDeletionVectors'        = 'true',
  'delta.enableChangeDataFeed'         = 'true',
  'delta.deletedFileRetentionDuration' = 'interval 90 days',
  'delta.logRetentionDuration'         = 'interval 90 days'
)
COMMENT 'Grain: one row per account per fund holding per valuation date.';
```

AUDIT/RESTATEMENT_DELTA.SQL — SQL

```sql
-- what did the valuation equal when the statement was produced?
WITH issued AS (
  SELECT account_sk, sum(value_gbp) AS amt
  FROM   nexus.gold.fct_holding_valuation_daily
         TIMESTAMP AS OF '2026-07-01 06:00:00'
  WHERE  valuation_date = '2026-06-30' GROUP BY 1),
current_state AS (
  SELECT account_sk, sum(value_gbp) AS amt
  FROM   nexus.gold.fct_holding_valuation_daily
  WHERE  valuation_date = '2026-06-30' GROUP BY 1)
SELECT c.account_sk, i.amt AS issued, c.amt AS now,
       c.amt - coalesce(i.amt, 0) AS delta_gbp
FROM   current_state c LEFT JOIN issued i USING (account_sk)
WHERE  c.amt <> coalesce(i.amt, 0) ORDER BY abs(delta_gbp) DESC;
```

CONSEQUENCES, STATED PLAINLY

- **Storage cost is decoupled from the engine.** History stays at object-storage rates on ADLS Gen2 and can be queried directly.
- **Fabric needs no second copy.** Fabric reads Delta natively through OneLake, so moving between engines doesn't mean moving data (page 34).
- **Clustering has a price.** Liquid clustering ends the partition-design argument, but the keys must still cover most predicates, and the maintenance cost is reviewed weekly.

THREE OPERATIONAL USES OF TIME TRAVEL

- **Restatement evidence.** A late price correction or backdated transaction shifts a prior valuation. The per-account difference exists *before* client services are asked about it.
- **Regression testing in CI.** A candidate build is compared with the prior version; a certified measure that moves by more than 0.5% fails the merge.
- **Recovery without restore.** A bad deployment is reversed by re-running from a prior version, in minutes.

> **Limitation.** Time travel only answers within the history window. A statement older than 90 days is reproduced from the bitemporal satellites in silver (page 29). That is a large part of why I would put the vault there.

Data Vault in silver

Quilter has several businesses describing the same client. That's where I'd use Data Vault, and here's how.

MODELS/SILVER/VAULT_CLIENT.SQL — SQL · DELTA

```sql
-- hub: one row per business key, ever. Insert-only.
MERGE INTO nexus.silver.hub_client t
USING (
  SELECT DISTINCT
         sha2(concat_ws('||', upper(trim(client_ref)), bk_domain), 256)
                                AS client_hk,
         client_ref, bk_domain,
         current_timestamp()    AS load_ts,
         record_source
  FROM   nexus.bronze.stg_client          -- advice, platform, DFM feeds
) s
ON t.client_hk = s.client_hk
WHEN NOT MATCHED THEN INSERT *;

-- satellite: a new row only when the descriptive payload changes
INSERT INTO nexus.silver.sat_client_details
SELECT s.client_hk, s.load_ts, s.effective_from, s.hashdiff,
       s.title, s.surname, s.date_of_birth, s.email_address,
       s.national_insurance_no, s.record_source
FROM   nexus.bronze.stg_client_hashed s
LEFT JOIN (SELECT client_hk, hashdiff,
                  row_number() OVER (PARTITION BY client_hk
                                     ORDER BY load_ts DESC) AS rn
           FROM nexus.silver.sat_client_details) cur
       ON cur.client_hk = s.client_hk AND cur.rn = 1
WHERE  cur.hashdiff IS NULL OR cur.hashdiff <> s.hashdiff;

-- link: client ↔ account ↔ adviser firm, many-to-many by design
-- hub_account, hub_adviser_firm, link_client_account_adviser ...
```

WHY A VAULT IN SILVER, HERE

- **Several businesses describe the same client.** Financial planning, the investment platform, multi-asset and discretionary management each hold their own view. A hub integrates them by business key without forcing a conformed model before the definitions are agreed.
- **Insert-only history is audit evidence.** Every satellite row carries its load time and record source, so "what did we know, and from where, on that date" is a query. Complaints, remediation and the regulator all ask that question.
- **New sources are additive.** An acquired book or a new adviser back-office feed adds a satellite. It does not restructure a dimension that reports already depend on.
- **Kimball stays the consumption model.** Gold dimensions and facts are built from the vault (pages 30–31). Analysts never query hubs and links directly.

> **Limitation.** At Lifepal, silver was conformed Kimball with SCD2, and I know Data Vault at working level and haven't run it in production. This page is how I would apply it and where I would stop: no vault for a single-source domain, where it is ceremony without a payoff.

Conformed dimensions and SCD2

For reporting I use Kimball. A fund charge that changes in April shouldn't change what we told clients in March.

SRC/DIM_FUND_SHARE_CLASS_SCD2.PY — PYSPARK · DELTA

```
staged = (spark.table("nexus.silver.sat_fund_share_class_current")
    .transform(standard_asset_class)       # one vocabulary, all lines
    .withColumn("fund_nk", F.col("isin"))
    .withColumn("row_hash", F.sha2(F.concat_ws("||", *TRACKED), 256)))
    # TRACKED = ongoing_charge_pct, risk_rating, asset_class, share_class

target  = DeltaTable.forName(spark, "nexus.gold.dim_fund_share_class")
current = target.toDF().filter("is_current")

changed = (staged.alias("s").join(current.alias("t"), "fund_nk", "left")
           .filter("t.fund_nk IS NULL OR t.row_hash <> s.row_hash")
           .select("s.*"))

# two-pass SCD2: a null merge key forces the INSERT arm for the new version
close_rows  = changed.withColumn("_mergekey", F.col("fund_nk"))
insert_rows = changed.withColumn("_mergekey", F.lit(None).cast("string"))

(target.alias("t")
  .merge(close_rows.unionByName(insert_rows).alias("s"),
         "t.fund_nk = s._mergekey AND t.is_current")
  .whenMatchedUpdate(set={"valid_to": "s.effective_from",
                          "is_current": F.lit(False)})
  .whenNotMatchedInsert(values={
      "fund_sk":   F.expr("xxhash64(s.fund_nk, s.effective_from)"),
      "fund_nk":   "s.fund_nk",
      "ongoing_charge_pct": "s.ongoing_charge_pct",
      "risk_rating":        "s.risk_rating",
      "valid_from": "s.effective_from",
      "valid_to":   F.lit("9999-12-31").cast("date"),
      "is_current": F.lit(True)})
  .execute())
```

See the code: **github.com/rfim/nexus-quilter/src/nexus/silver.py**

WHERE THE DIFFICULTY ACTUALLY LIES

- **Definitions take the time.** At Lifepal, agreeing the predicate for a "sale" across commercial, finance and partner teams took four workshops and a written definition. The MERGE took an afternoon. The definitions are where the time goes, and they are what the role means by influencing non-technical stakeholders.
- Each source keeps its own staging and its own quirks. **Conforming happens in silver**, so source teams aren't asked to change their systems.
- **Surrogate keys are generated in the platform.** A provider renumbering its codes cannot corrupt historical joins.
- SCD2 on charges and risk ratings means a repriced share class cannot rewrite a prior period's cost disclosure. That is the most common cause of "the number changed since last week".

> **Wealth translation.** "Plan version" becomes share-class version and "partner" becomes fund manager or adviser firm. A client becomes an individual, a household or an account holder, depending on which business is asking. The mechanics don't change; only the vocabulary does.

Incremental fact table

This is the single change that saved the most time at Lifepal: merge only what's new, plus a lookback window sized from real data.

SRC/GOLD/FCT_PLATFORM_TRANSACTION.PY — PYSPARK · DELTA

```python
from datetime import datetime
from delta.tables import DeltaTable
from pyspark.sql import functions as F

FACT = "nexus.gold.fct_platform_transaction"
LOOKBACK = "INTERVAL 72 HOURS"   # p99.5 of observed late-arrival delay

def build_fct_platform_transaction(spark):
    hwm = (spark.table(FACT).agg(F.max("_ingested_at")).first()[0]
           or datetime(1900, 1, 1))                  # first run = full load

    src = (spark.table("nexus.silver.platform_transaction")
           .filter("NOT _deleted")
           .filter(F.col("_ingested_at") > F.lit(hwm) - F.expr(LOOKBACK)))
    fund = spark.table("nexus.gold.dim_fund_share_class")

    # point in time: the share-class version in force on the trade date
    fact = (src.alias("s").join(fund.alias("f"),
              (F.col("f.fund_nk") == F.col("s.isin"))
              & (F.col("s.trade_date") >= F.col("f.valid_from"))
              & (F.col("s.trade_date") <  F.col("f.valid_to")), "left")
             .select(F.xxhash64("s.transaction_id").alias("transaction_sk"),
                     "f.fund_sk", "s.account_sk", "s.client_sk",
                     "s.wrapper_sk", "s.trade_date", "s.transaction_type",
                     "s.amount_gbp", "s.units", "s.legal_entity",
                     "s._ingested_at"))

    (DeltaTable.forName(spark, FACT).alias("t")
       .merge(fact.alias("u"), "t.transaction_sk = u.transaction_sk")
       .whenMatchedUpdateAll()
       .whenNotMatchedInsertAll()
       .execute())
```

See the code: **github.com/rfim/nexus-quilter/src/nexus/gold.py**

THREE KEY PARAMETERS

- **The lookback comes from the data.** At Lifepal, 72 hours was the p99.5 of measured partner confirmation delay. At Quilter, the number would come from observed settlement and correction lag. The method transfers; the value is something I would measure.
- **Compute is scoped to the model.** This model alone justifies a larger cluster. The rest of the project runs on the smallest compute that meets its SLA.
- **Point-in-time joins.** A transaction reports against the share class as it was on the trade date, which removed a recurring source of dispute with finance.

MEASURED EFFECT AT LIFEPAL

- Median run time fell from **2 h 14 min** (full rebuild) to **11 min** (merge), measured across 30 runs either side.
- This one model accounts for roughly half of the batch-window reduction on page 03.

> **The trade-off.** An incremental model is harder to reason about than a full rebuild, and it can drift. That is why the CI metric diff on page 10 exists: the cost of incrementality is paid in tests, and it is worth paying.

Quality checks at ingestion

At Lifepal we ran 512 checks across 47 sources and kept the pass rate above 98%. Bad rows go to quarantine, where someone owns them.

PIPELINES/PLATFORM_TRANSACTION_SDP.PY — DECLARATIVE PIPELINE

```python
import dlt          # Lakeflow Spark Declarative Pipelines
from pyspark.sql import functions as F

RULES = {
  "transaction_id_present": "transaction_id IS NOT NULL",
  "type_known":   "transaction_type IN ('contribution','withdrawal',"
                  "'switch_in','switch_out','fee','income',"
                  "'transfer_in','transfer_out')",
  "isin_shape":   "isin RLIKE '^[A-Z]{2}[A-Z0-9]{9}[0-9]$'",
  "wrapper_known": "wrapper_code IN ('ISA','JISA','PENSION','GIA','BOND')",
  "amount_nonzero":  "amount_gbp <> 0",
  "trade_not_future": "trade_date <= current_date()",
}

# Clean rows continue; failures are kept, annotated and inspectable.
@dlt.table(name="platform_transaction_valid",
           table_properties={"quality": "silver"})
@dlt.expect_all_or_drop(RULES)
def platform_transaction_valid():
    return dlt.read_stream("platform_transaction_raw")

@dlt.table(name="platform_transaction_quarantine",
           comment="Rejected rows with the failing rule attached.")
def platform_transaction_quarantine():
    failing = " OR ".join(f"NOT ({e})" for e in RULES.values())
    return (dlt.read_stream("platform_transaction_raw").filter(failing)
            .withColumn("_failed_rules", F.array_compact(F.array(*[
                F.when(F.expr(f"NOT ({e})"), F.lit(n))
                for n, e in RULES.items()])))
            .withColumn("_quarantined_at", F.current_timestamp()))
```

See the code: **github.com/rfim/nexus-quilter/src/nexus/silver.py**

QUARANTINING BAD ROWS

- A failing row is written to a quarantine table with the failing rule attached, so the failure stays **inspectable** and no partial data reaches a data product.
- Silently accepting malformed data is by far the costlier failure. This design removes it in exchange for a small storage overhead, a trade I would make every time.
- The quarantine table has an owner in the contract and a weekly review. Without both, it quietly becomes a landfill.

THE SAME RULES, TWICE

- **At ingestion**, as expectations, where rejection is cheapest.
- **At the model**, as Python model contracts and referential checks inside the job. A referential failure stops the build before an orphaned fact is published.

> **Signed off by the owner.** The rule set is versioned with the model and signed off by the data owner. When the definition of a valid transaction changes, the diff shows who agreed to it and when. That makes the data product auditable.

Metric definitions

Most "two numbers in one meeting" problems come from copied definitions. I keep each definition in one place.

SEMANTIC/MV_CLIENT_FLOWS.SQL — METRIC VIEW

```sql
CREATE VIEW nexus.gold.mv_client_flows
WITH METRICS LANGUAGE YAML AS $$
version: 0.1
source: nexus.gold.fct_platform_transaction
joins:
  - name: wrapper
    source: nexus.gold.dim_wrapper
    on: source.wrapper_sk = wrapper.wrapper_sk
  - name: client
    source: nexus.gold.dim_client
    on: source.client_sk = client.client_sk
dimensions:
  - name: Wrapper              # synonyms feed natural-language querying
    expr: wrapper.wrapper_name
  - name: Distribution Channel
    expr: client.distribution_channel
  - name: Legal Entity
    expr: source.legal_entity
  - name: Month
    expr: DATE_TRUNC('MONTH', source.trade_date)
measures:
  - name: Gross Inflows
    expr: SUM(CASE WHEN amount_gbp > 0 THEN amount_gbp END)
  - name: Gross Outflows
    expr: -SUM(CASE WHEN amount_gbp < 0 THEN amount_gbp END)
  - name: Net Flows                      # certified. One definition.
    expr: SUM(amount_gbp)
  - name: Active Clients
    expr: COUNT(DISTINCT source.client_sk)
$$;
```

ONE OBJECT, THREE CONSUMERS

- **Reporting.** Power BI and Fabric semantic models compose the certified measure; none re-expresses it in DAX. Only presentation logic lives downstream.
- **Natural-language querying.** Questions resolve against the same measures and filters, so a chat answer and a dashboard cannot disagree.
- **Notebooks.** Analysts read the certified measure directly, without rebuilding it in pandas, which is where most "two numbers, one meeting" problems start.

THE INTERFACE CONTRACT WITH REPORTING

- A new business definition goes into the metric view **first**, then into the report.
- Grain, owner and freshness are declared with the metric, so a report author knows what they can slice before building the visual.
- At Lifepal, the **60% fall in ad-hoc requests** is attributed to this release: the trailing 90-day count went from 214 to 86 over the next two quarters.

> **Limitation.** Net Flows is additive and safe to sum. AUA is a point-in-time balance, so it lives on the valuation snapshot and is always read at a single date. Making that distinction explicit in the catalog is half the value of a metric layer.

Databricks and Fabric

How I'd see Databricks and Fabric working together at Quilter, on the same Delta tables.

ADLS Gen2

One copy of bronze, silver and gold as Delta, in a storage account Quilter owns.

`Delta · Unity Catalog`

→

Databricks

Engineering, streaming, data quality, ML. Writes gold and certifies metrics.

`Workflows · SDP · MLflow`

→

OneLake

Reads gold in place through a shortcut or a mirrored Unity Catalog. No copy job.

`Shortcuts · mirrored catalog`

→

Power BI

Direct Lake semantic models over certified gold. Business logic stays in the metric layer.

`Semantic models · Direct Lake`

PRINCIPLES I WOULD HOLD

- **One copy, two engines.** Fabric reads governed Delta tables and doesn't ingest data separately. A table that exists in both places as copies is a reconciliation problem with a start date.
- **Engineering lives in one place.** Transformation, quality and certification run in Databricks. Fabric holds semantic models and presentation, which is where its strengths are.
- **Real-Time Intelligence for operational views.** Intraday platform events can feed an Eventhouse for operational dashboards, while the same stream lands in bronze for the governed record.
- **Administration is part of the design.** Workspaces per domain, capacity sized and monitored like cluster policy, and deployment pipelines for semantic models alongside Asset Bundles.

QUESTIONS I WOULD SETTLE EARLY

- Which tables carry Unity Catalog row filters or masks, and what is the supported path for them into Fabric today? I would test this before relying on it.
- Where does a Power BI measure end and a metric-view measure begin? One rule, written down, avoids two definitions of Net Flows.
- Who owns capacity cost, and is it attributed to domains the way cluster spend is (page 36)?

> **Limitation.** My production experience is Databricks and Delta; Fabric I know at concept level. Because Fabric reads Delta natively, the engineering carries over directly. Capacity management and Fabric administration are what I would learn from the team.

lifepal

AI agents on governed data

I'm keen on AI, but agents should only see what the person asking can see. Here's how I'd set that up.

AI/TOOLS.SQL — UC FUNCTIONS AS TOOLS

```sql
-- A tool is a governed function. No DDL, no DML, no service principal.
CREATE OR REPLACE FUNCTION nexus.ai.certified_metrics(domain STRING)
RETURNS TABLE (metric STRING, definition STRING,
               owner STRING, grain STRING, freshness_minutes INT)
COMMENT 'Metrics an agent is permitted to quote, with definitions.'
RETURN SELECT metric, definition, owner, grain, freshness_minutes
       FROM nexus.gold.metric_catalogue WHERE domain_name = domain;

GRANT EXECUTE ON FUNCTION nexus.ai.certified_metrics TO `grp_analyst`;
```

AI/RETRIEVAL.PY — PYTHON

```python
from databricks.vector_search.client import VectorSearchClient

index = VectorSearchClient().get_index(
    endpoint_name="nexus-vs", index_name="nexus.ai.fund_literature_idx")

hits = index.similarity_search(
    query_text=question, num_results=8,
    columns=["chunk", "source_url", "doc_version"],
    # filter mirrors the row filter, so retrieval cannot exceed the
    # caller's grants. This is the whole control.
    filters={"legal_entity": ctx.legal_entities})["result"]

answer = spark.sql(
    "SELECT ai_query(:endpoint, :prompt)",
    args={"endpoint": SERVING_ENDPOINT,
          "prompt": GROUNDED_PROMPT.format(ctx=cite(hits), q=question)}
).first()[0]
```

FOUR DESIGN COMMITMENTS

- **No additional data processor.** No client text leaves the workspace, so the privacy assessment needs no new vendor review, which is often what takes longest.
- **Retrieval inherits authorisation.** The search filter comes from the same grant table as the row filter, so an agent cannot retrieve a document its caller could not read.
- **Citation is mandatory.** Every answer returns its source document and version. An answer without a citation is logged as a failure.
- **Evaluated against a fixed set.** A reference question set is scored nightly; a fall in grounded accuracy blocks promotion of the model version.

My position. Agents need context, and the lazy route to context is a broadly privileged service principal. That route turns every control on pages 19–22 into decoration. In a wealth business there is a second line: an internal agent answers questions about data and literature, and it never gives advice.

Already in practice. At Lifepal I contributed to an internal MCP server that exposes Databricks and Airflow to AI agents, and I use Cursor and Claude Code every day.

Managing platform cost

Cost was one of my favourite problems at Lifepal. Most of the saving came from refreshing data only as often as people needed it.

PLATFORM/POLICIES.JSON — CLUSTER POLICY

```
{
  "custom_tags.owning_team": {"type": "unlimited", "isOptional": false},
  "custom_tags.workload":    {"type": "allowlist",
                              "values": ["ingest","transform","bi","ml"]},
  "autotermination_minutes": {"type": "range", "maxValue": 20,
                              "defaultValue": 10},
  "spark_version":           {"type": "regex", "pattern": ".*-lts-.*"},
  "node_type_id":            {"type": "allowlist",
                              "values": ["Standard_D8ds_v5","Standard_E8ds_v5"]},
  "azure_attributes.availability":
                             {"type": "fixed", "value": "SPOT_WITH_FALLBACK_AZURE"},
  "num_workers":             {"type": "range", "maxValue": 8}
}
```

FINOPS/ATTRIBUTION.SQL — SQL

```sql
-- spend resolved to team and workload. Published weekly, to the teams.
SELECT u.custom_tags.owning_team, u.custom_tags.workload,
       round(sum(u.usage_quantity * p.pricing.effective_list.default), 0)
                                                    AS spend_90d,
       round(100 * ratio_to_report(sum(u.usage_quantity *
            p.pricing.effective_list.default)) OVER (), 1)  AS pct
FROM   system.billing.usage u
JOIN   system.billing.list_prices p
       ON  u.sku_name = p.sku_name
       AND u.usage_end_time BETWEEN p.price_start_time
           AND coalesce(p.price_end_time, current_timestamp())
WHERE  u.usage_date >= current_date() - INTERVAL 90 DAYS
GROUP BY ALL ORDER BY spend_90d DESC;
```

FOUR MECHANISMS

- **Isolation.** Each workload has its own compute, which removes cross-workload contention and makes every unit of spend attributable to one team.
- **Mandatory tags in policy.** Attribution only works if it cannot be skipped. The policy refuses a cluster without an owning team.
- **Auto-termination, spot with fallback, and serverless where the shape suits it.** Idle compute is the cheapest saving available and the easiest one to forget.
- **Budget alerts acted on weekly**, in the same meeting as incidents and delivery, so a cost regression is handled as fast as a defect.

FINDINGS FROM ATTRIBUTION, AT LIFEPAL

- A dataset refreshing every 15 minutes fed a report opened twice a week. Moving it to six-hourly **saved more than every query optimisation that quarter combined**.
- A small cluster never terminated because a stale BI connection held it open. Attribution found it; a shorter auto-termination fixed it.
- Publishing per-team cost changed behaviour with no policy change at all: once the figure had their name on it, teams optimised their own models.

CLOSING · TRANSITION · A NEW ROLE

My first 90 days

If I join, this is roughly how I'd spend my first three months.

PHASE	WHAT IT IS	THE ARTEFACT IT LEAVES BEHIND	THE FAILURE IT PREVENTS
01 · Listen & inventory Days 1–30	Learn the domain from its owners and its data. Build a queryable record of what exists, what it feeds and who owns it, using catalog lineage. Nothing gets changed in this phase.	An `object_owner` table, populated ownership tags, and a lineage view that answers "what breaks?" before anyone touches a model.	Standardising an estate you do not understand yet.
02 · Own Days 31–60	Take the highest-traffic data products first. Run them, fix them and join their incident rota. Write contracts for their sources and expectations for the business rules that currently live in someone's head.	Contracts and expectations for those sources, and a run-book that has actually been used in an incident.	Ownership on paper only. If you have never recovered the pipeline at 7am, it isn't yours yet.
03 · Standardise Days 61–90	Agree the patterns with the other leads and the architects: one ingestion factory, one quality mechanism, one deployment path, adopted where each has proved itself twice.	Standards written as code and CI gates, a community-of-practice session, and a roadmap with risks and dependencies named.	Permanent drift: three retry idioms and two alerting paths, compounding.

Write knowledge down

A handover meeting evaporates. A contract file with a named owner, a test that encodes the rule, and a model description that fails CI when it is missing will all outlast it. Every knowledge-sharing session should end with a merged pull request, or it didn't really happen.

Questions I would bring to the conversation

- Which domains would this role lead first, and what does a good first year look like for them?
- Is Unity Catalog already the governance plane across the estate, or is that still in progress?
- Where does Fabric sit today: serving only, or engineering as well?
- How is Consumer Duty outcome MI produced now, and who owns its definitions?

Summary

Thank you for reading. Every figure here traces to a run or a query, and I'm happy to go as deep as you like on any of them.

Open substrate

Delta on ADLS Gen2 under Unity Catalog, read in place by Fabric. Switching engines doesn't mean moving data.

Governed by default

Governed tags, column masks, legal-entity row filters, column lineage and scheduled monitoring. Protection is in place before the first query.

Continuously costed

Isolation, mandatory tags, attribution, and refresh cadence matched to SLA: $11k against a $19k projection, held for four quarters.

Auditable over time

Idempotent merge, an insert-only vault, SCD2 and time travel. What we knew, and when, is a query.

Reproducible practice

140 pipelines from one reviewed module, onboarding halved, two engineers promoted, and the First Light playbook. Each part of the role can now be tried live.

Quilter's values, with examples

Do the right thing
Limitations are flagged wherever the mechanism differs from production or my experience is working-level.

Always curious
Unity Catalog ABAC, Fabric and Data Vault taken from working knowledge toward production.

Embrace challenge
Batch window 6 h → 90 min; reporting latency 24 h → under 5 min.

Stronger together
Two promotions, onboarding halved, and 100+ engineers taught outside work.

Production experience and working knowledge

In production I have run Databricks within Azure: Delta on ADLS Gen2, Workflows, ADF, Key Vault, Spark tuning, Kafka and Debezium CDC, PySpark models, Great Expectations and Terraform. Unity Catalog, Asset Bundles and declarative pipelines I know at working level, and they are shown here as I would build with them. Data Vault and Fabric are marked on their own pages. Modelling, idempotence, quality and delivery practice transfer unchanged.

R. Firman Insan Muhammad (VIM) · Lead Data Engineer · London · firman.insan.m@gmail.com · right to work in the UK, no sponsorship required. Thank you for your time.

See how I work, live: **rfim.github.io/nexus-quilter/simulate.html** · code: **github.com/rfim/nexus-quilter**